Inspiring Excellence

CSE340

Summer 2025

Assignment 3

Name : Abdullah Al Fahad
ID : 21201240
Section : 8
Faculty : PBK

Ans to the Ques no 1

0_x ABB 9609

1010 1011 1011 1001 0110 0000 1001

0_xABB9609 ⇒ 1010 1011 1011 1001 0110 0000 1001

1 bit	6 bit	21 bit
Sign bit	Exponent bit	fraction bit
1	010101	11011 1001 0110 0000 1001

Biased exp ⇒ 010101 ⇒ 21

Bias ⇒ $2^{n-1}-1$ ⇒ $2^{6-1}-1$ ⇒ 31

Actual Exponent ⇒ 21 − 31 ⇒ −10

Fraction ⇒ 0.11011 1001 0110 0000 1001

⇒ 0.862064838 4

Decimal ⇒ $(-1)^1 \times (1+0.8620648384) \times 2^{-10}$

⇒ $-1.8620648384 \times 2^{-10}$

⇒ $-1.818422694 \times 10^{-3}$

Ans

Ans to the Ques no 2

a) 50.7869 ⇒ 110010.11001

⇒ 1.10010110001×2^5

79.83 = 1001111.11010

⇒ 1.00111111010×2^6

29.58 ⇒ 11101.10010

⇒ $1.11011001 0 \times 2^4$

1st Part

~~50.7869 - 29~~

79.83 − 29.58

⇒ $1.00111111010 \times 2^6 - 1.11011001 0 \times 2^4$

⇒ $1.00111111010 \times 2^6 - 0.01110110010 \times 2^6$

⇒ 0.10001001000×2^6

⇒ 1.00010010000×2^7

.7869 ×2	
1	.3738 ×2
1	.1476 ×2
0	.2952 ×2
0	.5904 ×2
1	.1808 ×2
0	.3616 ×2
0	.7232 ×2
1	.4464

.58 ×2	
1	.16 ×2
0	.32 ×2
0	.64 ×2
1	.28 ×2
0	.56

0.83 ×2	
1	.66 ×2
1	.32 ×2
0	.64 ×2
1	.28 ×2
0	.56

2nd Part

$50.7969 + (7983 - 2958)$

$\Rightarrow 1.10010111001 \times 2^5 + 1.0001001000 \times 2^7$

$\Rightarrow 0.01100101110011 \times 2^7 + 1.0001001000 \times 2^7$

$\Rightarrow 1.0111\ 0111\ 1001 \times 2^7$

(b) $64.2486 \Rightarrow 1000000.00111$

$\Rightarrow 1.0000\ 0000\ 111 \times 2^6$

$49.1832 \Rightarrow 110001.00101$

$\Rightarrow 1.1000100101 \times 2^5$

$\Rightarrow 1.0000\ 0000\ 111 \times 2^6 \times 1.1000100101 \times 2^5$

$\Rightarrow (1.00000000111 \times 1.1000100101) \times 2^{6+5}$

$\Rightarrow 1.1000101010011000000011 \times 2^{11}$

.2486 ×2

0.	.4972 ×2
0	.9944 ×2
1	.9888 ×2
1	.9776 ×2
1	.9552

.1832 ×2

0	.3664 ×2
0	.7328 ×2
1	.4656 ×2
0	.9312 ×2
1	.8624

Ans to the Que no 3

$28.4810 \Rightarrow 11100.01111$

$\Rightarrow 1.1100\ 01111 \times 2^4$

$4.0210 \Rightarrow 100.00000$

$\Rightarrow 1.0000000 \times 2^2$

$28.4810 - (-4.0210)$

$\Rightarrow 28.4810 + 4.0210$

$\Rightarrow 1.1100\ 01111 \times 2^4 + 1.0000000 \times 2^2$

$\Rightarrow 1.1100\ 01111 \times 2^4 + 0.0100000000 \times 2^4$

$\Rightarrow 10.0000\ 0111\ 1 \times 2^4$

$\Rightarrow 1.0000\ 0011\ 11 \times 2^5$

In Single Precision Exponent has 8 bit

Range of Biased Exponent

$\Rightarrow 0$ to $(2^8 - 1)$

$\Rightarrow 0$ to 255

$\Rightarrow 1$ to 254

.4810 × 2

0	.962 × 2
1	.924 × 2
1	.848 × 2
1	.696 × 2
1	.392

.0210 × 2

0	.042 × 2
0	.084 × 2
0	.168 × 2
0	.336 × 2
0	.672

Now, Bias $\Rightarrow 2^{8-1}-1 \Rightarrow 127$

Bias Exponent $\Rightarrow 127+5$

$\Rightarrow 132$

which is in the range $\therefore$ No overflow/underflow occured.

Ans to the Ques no 4

a) In IEEE754 floating point representation the exponent field is stored as an unsigned integer but in reality exponent can be positive or negative to handle both positive and negative in a uniform way; IEEE 754 uses a bias.

Biased exponent $\Rightarrow$ actual exponent + bias

Now, In single precision bias $= 2^{n-1}-1 \Rightarrow 2^{8-1}-1 \Rightarrow 127$

If actual Exponent $=5$; Biased exponent $\Rightarrow 5+127$
$\Rightarrow 132$

OR,
If actual exponent $= -5$; Biased exponent $\Rightarrow -5+127$
$\Rightarrow 124$

So, both positive and negative exponents are both neatly encoded as unsigned integers without needing a seperate "sign bit" for the exponent. This make hardware simpler and comparison easier.

(b) Optimized multiplication improved efficiency and performance in various ways. ~~Find it use.~~
In optimize multiplication it use only two register instead of three register.
In Optimized multiplication ALU is 64 bit where in traditional multiplication ALU size is 128 bit so, 64 bit calculation is faster than 128 bit calculation. Moreover, Traditional long multiplication uses 3 three register, where optimized multiplication uses two register which make optimized multiplication more optimal and simpler hardware design.

Ans to the Ques no 5

i) mul: Do signed x signed multiplication, stores lower 64 bits ~~lower~~ of the product.

Syntax: mul rd, rs1, rs2

Ex: addi x5, x0, 10
add9 x6, x6, -3
mul x7, x5, x6 // x7 = -30

ii) mulh: stores the uppen 64 bits of the 128 bit product. Do signed x signed multiplication and It use to detect overflow ing signed 64 bit multiplication.

Syntax: mulh rd, rs1, rs2

Ex: mulh x7, x5, x6 // x7 = uppen 64 bit of product.

iii) mulhu: It does unsigned x unsigned multiplicatio stores the upper 64 bit of the 128 bit product

Syntax: mulhu rd, rs1, rs2
x5 => 0x FFFFFFFFFFFFFFFF
x6 => 0x2

mulhu x_7, x_5, x_6 // x_7 = Upper 64 bits of unsigned product.

(4) mulhsu: multiply high signed/unsigned, gives the upper 64 bits of the product assuming one operand is signed and the other unsigned.

Syntax: mulhsu rd, rs1, rs2

Ex: $x_5 = -5000$ $x_6 = 4000$

mulhsu x_7, x_5, x_6

Ans to the Ques 6

i) Comparison instruction like fle.d stores boolean result (0/1) which is an integer value that's why it stores in integer register ~~which is x_5~~ Because x_5 stores integer value that's why x_5 is an integer register.

ii) False,

Single precision one 32 bits and fle.d here, (.d) denotes double precision which is 64 bits so, f_3 and f_5 both are double precision floating point number.

iii) fle.d x_5, f_3, f_5

it checks if $f_3 \leq f_5$ if true then $x_5 = 1$ or $x_5 = 0$

Now, if $x_5 = 0$; that's mean $f_3 > f_5$

Ans to the Ques no 7

```
feq.d x5, f1, f2
Beq x0, x5, JumpNotEqual
JumpEqual:
Beq x0, x0, Exit

JumpNotEqual:

Beq x0, x0, Exit

Exit:
```

Ans to the Ques 8

```
fle.d x5, f1, f2        // f1 ≤ f2
Beq x0, x5, JumpGreaten

JumpNotGreaten:
                        // or X19 = C
1 srli X19, X19, 4      // X19 = X19/4
Beq x0, x0, exit

JumpGreaten:

Exit:
```